<- This means this lab is on your local computer

# Lab 3: RISC-V, Venus

61C Summer 2026

# Announcement

- Exercise 5 is **optional**, since it covers content we haven't covered in lecture

# Please do this lab on your local machine!

This lab introduces Venus, and you will need to download the tools to run Venus on your local machine.

Have the reference card (https://cs61c.org/sp26/pdfs/resources/reference-card.pdf) next to you!

# Opening Venus

- Spec has detailed instructions
- Memcheck
- Demo time

# Some Venus Simulator Functions

- You will become very good friends with the simulator tab
- You can run from the start, step through code, reset the simulator
- You can set breakpoints in simulator by clicking on an instruction line
  - Or by putting ebreak before a line you wanna break on in your code
- You can view the values in registers or memory on the right side
  - You can change whether values are displayed in hex or decimal
- You can also go to a previous instruction  with the “prev” button
  - Especially helpful when you hit “step” too many times in a fit of range and zoom past the part you wanted to look at
- Demo time

# Arithmetic (R/I type instructions)

- Operates on register values (rs1, rs2) and store result back into a register (rd)

instr rd, rs1, rs2

- rd: destination register
- rs1: register 1
- rs2: register 2

ex: add, sub, mul, div, and, or, xor

instr rd, rs1, imm

- rd: destination register
- rs1: register 1
- Imm: immediate

ex: addi, ori, andi, xori

# Arithmetic: Examples

```
add x4, x3, x2    # x4 = x3 + x2
sub x10, x11, x12 # x10 = x11 - x12


and x25, x26, x27 # x25 = x26 & x27 (bitwise AND)
or x29, x30, x31  # x29 = x30 | x31 (bitwise OR)

addi x5, x6, 20   # x5 = x6 + 20 (with immediate)
ori x10, x11, 12  # x10 = x11 | 12 (bitwise OR with immediate)
```

# Conditionals (B type instructions)

- Allows program to jump to a different part of the code based on a condition
  - Conditions: Comparison result, Value of a register

instr rs1, rs2, label

- rs1: register 1
- rs2: register 2
- Label: represents where the program jumps to (target address)
  - This could also just be an immediate (number) which would make the program counter go up by that many bytes

ex: beq, bge, bgeu, blt, bltu

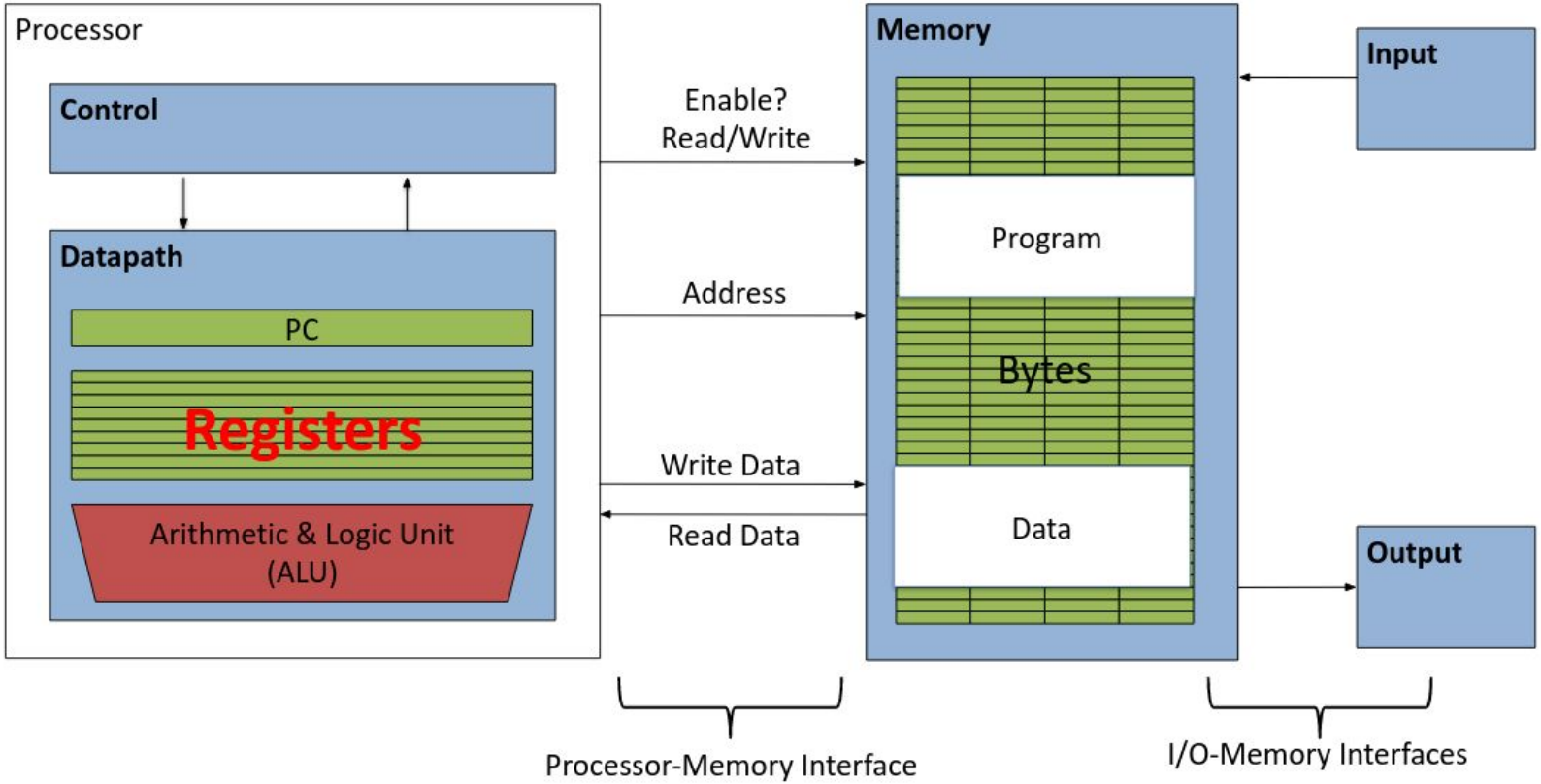
# Conditionals: Examples

```
#checks if x4 is equal to x3

beq x4, x3, J1


....

....

# code later executed if x4 = x3

J1:

...
```

# Pointers/Memory (Load/Save Instructions)

- w=word; h=halfword; b=byte
  - One word is 4 bytes
- Load instructions (lw/lh/lb)
  - lw rd imm(rs1)
  - (rs1 + imm) is a pointer to some data in memory, rd is the register you want to load into
- Save instructions (sw/sh/sb)
  - sw  rs2 imm(rs1)
  - Stores the contents of rs2 into the destination at address (rs1 + imm) in memory

# From lecture: registers are inside the processor!



# RISC-V Review

```
1.  .globl example # allows other files to find example
2.  .data          # specifies data/static segment
3.  n: .word 12    # n = var name, .word = size, 12 = value
4.  la t0, n       # loads address of n into t0
5.  lw t1, 0(t0)   # loads value in address at n
```

**C version:**

```
1.  t0 = &n;
2.  t1 = *t0;
```

# Directives (.data / .text)

- Assembly language instructions that tell the assembler how the program should be assembled
- Used to describe the type of data defined in a program
  - `.data` - defines global variables
  - `.text` - defines the code section of a program with instructions for the processor

# Ecall (don’t worry about this)

- Makes a system call to the operating system (OS) telling it to perform a specific task
- Examples
  - Allocating a block of memory
  - Printing a string
  - Opening a file